

SENIOR PROJECT [NAME]

SOFTWARE REQUIREMENTS SPECIFICATION



A. DEPARTMENT OF COMPUTER SCIENCE

Forman Christian College (A Chartered University)

TITLE PAGE

- B. Project Title
- C. Project Adviser Name
- D. Project Secondary Adviser Name
- E. Project Team
- F. Submission Date
- G. Signatures of Project Adviser and Project Secondary Adviser
- H. Signatures of Team Members

Table of Contents

Table of Contents	iii
Revision History	iv
1. Introduction.....	1
1.1 Review of Related Literature	1
1.2 Problem Statement	1
1.3 Proposed Solution	1
1.4 Problem Scope	1
1.5 Challenges	1
1.6 Knowledge Areas Required	1
1.7 Completeness Criteria	1
1.8 Research Outcomes/Nature of End Product	1
1.9 Learning Outcomes	1
1.10 Document Conventions.....	1
2. Background Study and Literature Survey	2
3. Overall Description.....	2
3.1 Proposed Solution	2
3.2 User Classes and Characteristics (if applicable)	2
3.3 Operating Environment	2
3.4 Design and Implementation Constraints	2
3.5 Assumptions and Dependencies	2
4. Functional Requirements	3
4.1 Requirement 1 OR Use-Case 1 (if requirements are modeled as use cases).....	3
4.2 Requirement 2 OR Use-Case 2 (and so on)	3
4.3 Proposed Workflow.....	4
4.4 Analysis and Modeling of Requirements	4
5. Nonfunctional Requirements	4
5.1 Target Performance.....	4
5.2 Safety Requirements (if applicable)	4
5.3 Security Requirements (if applicable)	4
5.4 Additional Software Quality Attributes	4
6. Other Requirements	4
7. Initial Results	5
8. Revised Project Plan.....	5
9. References	5
Appendix A: Glossary	6
Appendix B: IV & V Report.....	7
(Independent verification & validation).....	7

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Review of Related Literature

<Provide detailed review of related literature.>

1.2 Problem Statement

< Describe the problem for which the solutions would be provided.>

1.3 Proposed Solution

< Describe the proposed solution of the stated problem.>

1.4 Problem Scope

< Describe the scope of the problem that is covered by this SRS >

1.5 Challenges

1.6 Knowledge Areas Required

1.7 Completeness Criteria

1.8 Research Outcomes/Nature of End Product

1.9 Learning Outcomes

1.10 Document Conventions

<Describe any standards or typographical conventions that were followed when writing this SRS, such as fonts or highlighting that have special significance. For example, state whether italicized nouns represent external systems.>

2. Background Study and Literature Survey

<Provide understanding of the problem domain and detailed review of literature. Also provide limitations of existing work. Make headings as per requirement.>

3. Overall Description

3.1 Proposed Solution

<Summarize the major features the product contains or the significant functions that it performs or lets the user perform. Details will be provided in Section 3, so only a high level summary is needed here. Organize the functions to make them understandable to any reader of the SRS. Block diagram of the solution may be helpful. Flowchart may also be used.>

3.2 User Classes and Characteristics (if applicable)

<Identify the various user classes that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the favored user classes from those who are less important to satisfy.>

3.3 Operating Environment

<Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.>

3.4 Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

3.5 Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

4. Functional Requirements

<Functional requirements can be expressed as use-cases. Fill out the following template for each use-case if the requirements are modeled as use-cases. Otherwise write requirements one by one under each heading. Number every requirement. If modeling as use cases, do not name a use case as "Use-Case 1." State the use-case name like "Withdraw Cash from ATM". A use-case may have multiple alternate courses of action. >

<Provide a Use Case Diagram before describing the use cases.>

4.1 Requirement 1 OR Use-Case 1 (if requirements are modeled as use cases)

Identifier	UC-1	
Purpose	...	
Priority	<Choose one from {High, Medium, Low}>	
Pre-conditions	...	
Post-conditions	...	
Typical Course of Action		
S#	Actor Action	System Response
1		
2		
3		
...		
Alternate Course of Action		
S#	Actor Action	System Response
1		
2		
3		
...		

Table 1: UC-1

4.2 Requirement 2 OR Use-Case 2 (and so on)

<For example a requirement can be written as follows:

1. The proposed system should be able to perform tasks....>

4.3 Proposed Workflow

4.4 Analysis and Modeling of Requirements

<If applicable Include the following analysis models: use-case diagram (provide before Section 4.1), entity-relationship diagram, abstract class diagram, sequence diagram (to model system inputs and outputs, interaction between system and external world). Additional diagrams may be added for example flow chart, state diagram, data flow diagram (to model system inputs and outputs, interaction between system and external world), decision table, event table etc.>

5. Nonfunctional Requirements

5.1 Target Performance

<Provide target accuracy and efficiency etc. Other performance parameters may also be used. Describe those parameters and target values of those parameters.>

5.2 Safety Requirements (if applicable)

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

5.3 Security Requirements (if applicable)

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

5.4 Additional Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. These might include database requirements, external (hardware, software, or communication) interface requirements, internationalization requirements, legal requirements, and reuse objectives for the project.>

7. Initial Results

<Provide initial results obtained so far>

8. Revised Project Plan

<Provide project progress in accordance with the plan provided in project proposal. Gantt chart should be used in this regard. Use Microsoft Office to develop the Gantt chart. Also provide an updated project plan.>

9. References

<List all books, conference papers, journal articles, websites, etc. used in preparing the content of this SRS. Provide enough information so that the reader could access a copy of each reference, including title, author, volume/edition number, page number(s), and publication year. Mention complete URLs for websites.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: IV & V Report

(Independent verification & validation)

IV & V Resource

Name

Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
...					

Table 2: List of non-trivial defects